



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN



LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA

EJERCICIO DE CLASE N° 06

NOMBRE COMPLETO: Velazquez Caudillo Osbaldo

N° de Cuenta: 320341704

GRUPO DE LABORATORIO: 02

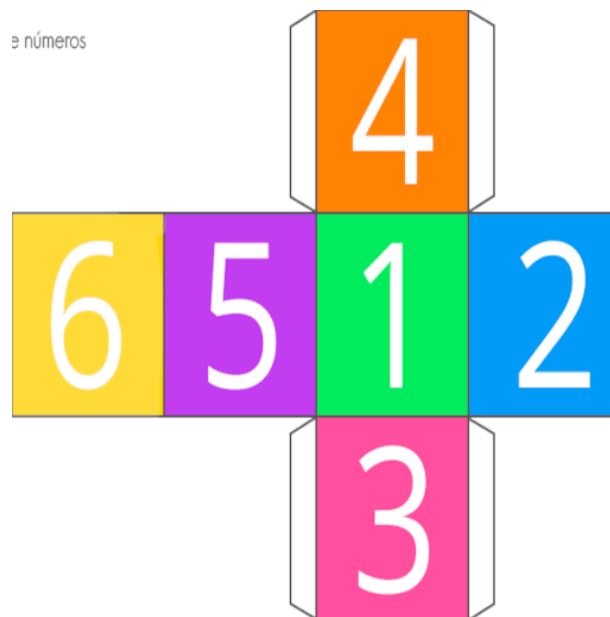
GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 07/10/25

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:



Esta es la imagen editada la cual redimensionamos de 512 x 512px además al número 6 le agregamos transparencia, toda la edición fue por medio del programa GIMP.

Esta será la imagen con la que trabajaremos tanto en Blender como en OPENGL

1. **Actividades realizadas.** Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Ejercicio 1: Texturizar su cubo con la imagen dado animales o dado números ya optimizada por ustedes

Para este ejercicio lo que hacemos es utilizar la imagen formato png, que se guardo en la carpeta Texture y lo impotamos en OPENGL.

```
datoTexture = Texture("Textures/dado-de-numeros-editado.png");  
datoTexture.LoadTextureA();
```

```
//Dado de OpenGL
//Ejercicio 1: Texturizar su cubo con la imagen dado_animales ya optimizada p
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-1.5f, 4.5f, -2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
dadoTexture.UseTexture();
meshList[4]->RenderMesh();
```

Las líneas de glEnable y glBlendFunc nos ayudan a que se vea la transparencia que le agregamos a nuestra imagen.

```
//Ejercicio 1: reemplazar con sus dados de 6 caras texturizados, agregar normales
// average normals
GLfloat cubo_vertices[] = {
    // front (5)
    //x      y      z      S      T      NX      NY      NZ
    -0.5f, -0.5f, 0.5f, 0.26f, 0.34f, 0.0f, 0.0f, -1.0f, //0
    0.5f, -0.5f, 0.5f, 0.49f, 0.34f, 0.0f, 0.0f, -1.0f, //1
    0.5f, 0.5f, 0.5f, 0.49f, 0.66f, 0.0f, 0.0f, -1.0f, //2
    -0.5f, 0.5f, 0.5f, 0.26f, 0.66f, 0.0f, 0.0f, -1.0f, //3
    // right (1)
    //x      y      z      S      T      NX      NY      NZ
    0.5f, -0.5f, 0.5f, 0.51f, 0.34f, -1.0f, 0.0f, 0.0f,
    0.5f, -0.5f, -0.5f, 0.74f, 0.34f, -1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, -0.5f, 0.74f, 0.65f, -1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 0.51f, 0.65f, -1.0f, 0.0f, 0.0f,
    // back (2)
    -0.5f, -0.5f, -0.5f, 0.99f, 0.34f, 0.0f, 0.0f, 1.0f,
    0.5f, -0.5f, -0.5f, 0.76f, 0.34f, 0.0f, 0.0f, 1.0f,
    0.5f, 0.5f, -0.5f, 0.76f, 0.65f, 0.0f, 0.0f, 1.0f,
    -0.5f, 0.5f, -0.5f, 0.99f, 0.65f, 0.0f, 0.0f, 1.0f,
    // left (6)
    //x      y      z      S      T      NX      NY      NZ
    -0.5f, -0.5f, -0.5f, 0.03f, 0.34f, 1.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, 0.5f, 0.24f, 0.34f, 1.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, 0.5f, 0.24f, 0.65f, 1.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, -0.5f, 0.03f, 0.65f, 1.0f, 0.0f, 0.0f,
    // bottom
    //x      y      z      S      T      NX      NY      NZ
    -0.5f, -0.5f, 0.5f, 0.51f, 0.0f, 0.0f, 1.0f, 0.0f,
    0.5f, -0.5f, 0.5f, 0.51f, 0.33f, 0.0f, 1.0f, 0.0f,
    0.5f, -0.5f, -0.5f, 0.74f, 0.33f, 0.0f, 1.0f, 0.0f,
    -0.5f, -0.5f, -0.5f, 0.74f, 0.0f, 0.0f, 1.0f, 0.0f,
    //UP
    //x      y      z      S      T      NX      NY      NZ
    -0.5f, 0.5f, 0.5f, 0.51f, 0.99f, 0.0f, -1.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 0.51f, 0.67f, 0.0f, -1.0f, 0.0f,
    0.5f, 0.5f, -0.5f, 0.74f, 0.67f, 0.0f, -1.0f, 0.0f,
    -0.5f, 0.5f, -0.5f, 0.74f, 0.99f, 0.0f, -1.0f, 0.0f,
};
```

Después modificamos los vértices esto para que el cubo tenga las texturas en el lugar correcto tomando en cuenta que va de 0 a 1 en ambos ejes y que la imagen tiene tamaños iguales al ser un cubo y las caras sean cuadrados.

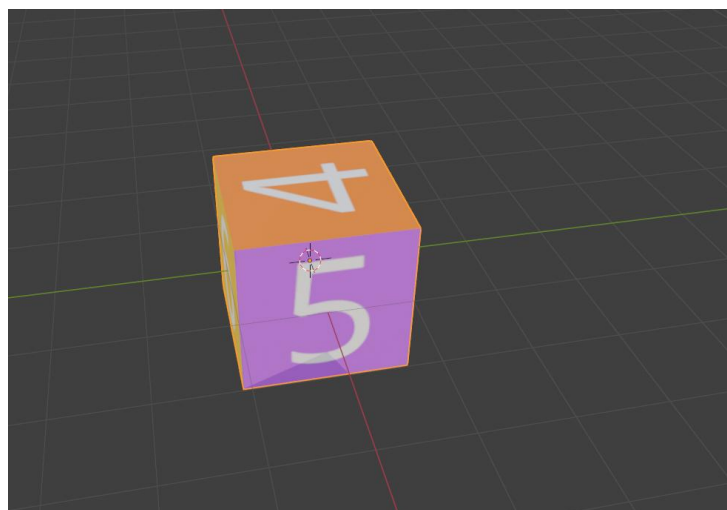
Ejecución



Ejercicio 2: Importar el cubo texturizado en el programa de modelado con la imagen dado_animales o dado_números ya optimizada por ustedes

Con ayuda de Blender vamos a texturizar el cubo, para ello nos apoyamos del cubo que ya esta por defecto en Blender al abrir el programa, para que quedara acomodado de manera correcta tenemos que aplicar escalas y rotaciones a cada una de las imágenes y acomodarlas en cada cara del cubo.

Se tomo como referencia el orden del cubo de OPENGGL para acomodar las caras del cubo de Blender



Ahora ya teniendo todo acomodado lo exportamos en formato obj y solo lo importamos

```
LogofiTexture.LoadTextureA();  
Dado_M = Model();  
Dado_M.LoadModel("Models/dado-colores.obj");
```

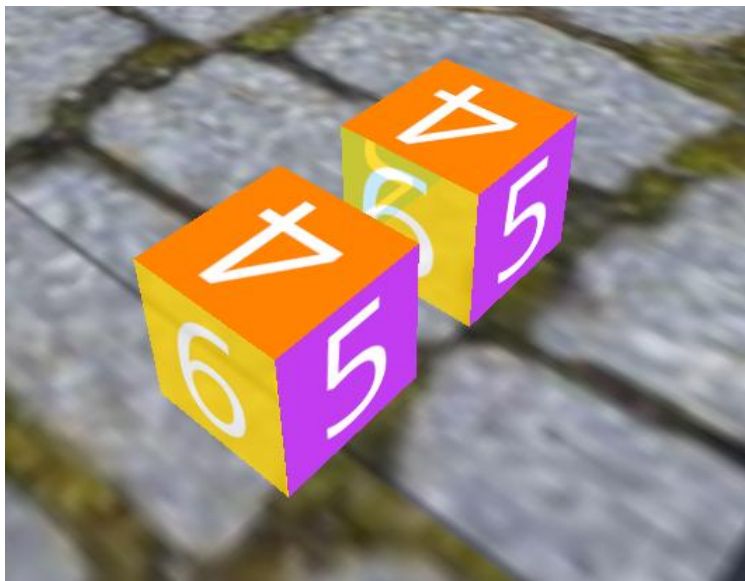
Este archivo se encuentra dentro de la carpeta Models

```
//Ejercicio 2: Importar el cubo texturizado en el programa de modelado con  
//la imagen dado_animales ya optimizada por ustedes  
  
//Dado importado  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(-3.0f, 4.5f, -2.0f));  
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));  
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Dado_M.RenderModel();
```

Lo acomodamos para que quede a la misma altura del otro cubo y lo rotamos para tener la misma cara en ambos cubos con una rotación de 90° grados



Ejecución



2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código

No se presentaron problemas

3. Conclusión:

La practica fue interesante sobre todo por que ya vemos como podemos texturizar a un objeto plano con ayuda de imágenes externas, lo que es de mucha ayuda para armar modelos propios, se me hizo más fácil con ayuda de Blender aunque se debe de tener cuidado con la configuración de

exportación del archivo. De igual manera los videos para el uso de GIMP fueron de mucha ayuda.